QUANTIZATION · APPLE SILICON · 2026

Data-Free Vector Quantization Beats Affine Quantization at Matched Bytes Below 6 Bits

Noah Zelezny

August 2026 · doi:10.5281/zenodo.22136000. Every number traces to a committed entry in the laboratory record; every margin is stated against a measured fit-to-fit noise floor for its geometry.

Abstract

Memory, not compute, is the binding constraint on local inference of large language models: a model is runnable on a given machine only if its weights fit in that machine's RAM, and for consumer and workstation hardware — 32 to 192 GB of unified memory — this excludes, at full precision, nearly every model approaching frontier performance. The gap is closed by quantization, the family of techniques that store each weight in a few bits rather than sixteen. The prevailing approach, affine quantization, rounds each weight independently onto a uniform grid; its most refined variants tune the grid per-channel on a calibration corpus, preserving more of the original model per gigabyte of stored size. This paper evaluates an alternative: **vector quantization (VQ)**, in which weights are grouped into vectors of d consecutive values — d is the *dimension* — and each vector is stored as an index into a *codebook*: a table of K representative d-dimensional vectors learned by k-means over the weight matrix itself. Storage cost is log2(K)/d bits per weight, and the method is entirely data-free: no calibration corpus, no activations, no teacher model.

We construct VQ quantizations of three models — Qwen3.5-397B-A17B and Qwen3.6-35B-A3B (mixture-of-experts), and Qwen3.8-27B (dense) — and compare them against both uniform and hand-tuned mixed-bit-depth affine builds of the same models, at matched or smaller file sizes. Each model is scored on one deterministic instrument throughout. For the two smaller models this is KL divergence: how far the quantized model's next-token probabilities drift from the full-precision original's, reported in millinats (defined in §2.6; zero means identical behavior). The 397B's full-precision teacher is too large to run on any machine we could assemble, so that model is scored by perplexity on frozen prose and code corpora — a measure against text rather than against the teacher. Three findings. **First, below approximately 5 bits per weight, the data-free VQ builds outperform the affine builds.** On the 397B model, a 1.75-bit-per-weight VQ build outscores the leading community build — a hand-tuned mixed-precision artifact at 2.6 bits per weight — on prose perplexity while being 19.6 GiB smaller, and ties it on code; a 2.25-bit VQ build beats the same comparator on both corpora at near-matched size, by 24 times the measurement noise on prose. On the 35B model, a 3.5-bit VQ build reaches 47.5 millinats at 16.6 GiB, where the community 4-bit affine build measures 78.6 millinats — about 31 millinats more — at 19.0 GiB. On the dense 27B, two VQ builds straddle the 4-bit affine conversion's size: the smaller (14.5 GiB) beats it by 12% KL divergence while being 0.5 GiB smaller, and the larger (15.5 GiB, 3.3% larger) beats it by 28%. The advantage has a measured boundary: on both architectures the affine frontier overtakes VQ in the 4.5-to-6 bit range — bracketed at 4.5–6.0 bpw on the dense model and 5.0–6.0 on the MoE — and at 8 bits affine quantization is essentially lossless, leaving nothing to improve upon. VQ's regime is the low-bit range — which is precisely the range in which large models fit on the hardware most people have. **Second, model size becomes continuously tunable**: a two-coefficient size model predicts an artifact's packed size to within a few tenths of a GiB before it is fit — validated on all three models against builds whose sizes were predicted before the builds existed — and a bit-harvesting technique reaches sizes between codebook steps at measured quality-per-byte exchange rates. **Third, weight-space reconstruction error — the statistic most quantization pipelines optimize and gate on — does not rank output quality in this regime.** In a pre-registered experiment, a fitter modification engineered to improve precisely the reconstruction statistic identified as decisive

produced a model 4.7 times worse than the one it was designed to improve. Only evaluation of the assembled model ranks artifacts.

Comparable behavior was observed on the gemma-4 model family, which is nonetheless excluded from all claims: raw likelihood is not a valid property of those instruction-tuned models, so no deterministic scoring scheme was available. The effect is likely broader than the three models studied; three models are what is claimed.

Thirteen artifacts spanning the three ladders are published with pinned revisions, and every comparison names the artifact and instrument that produced it.

---

1. Introduction

The feasibility of running a large language model locally is decided primarily by memory. Compute per token is modest for models with few active parameters — Qwen3.5-397B-A17B, the largest model studied here, activates 17 of its 397 billion parameters per token — but the weights must reside in RAM regardless of how many are active, and RAM is the commodity in shortest supply outside the datacenter. At 16-bit precision, a machine that holds such a model is unreachable without deliberate effort and a liberal budget. Quantization decides how low on the memory ladder a given model stays usable, and for the machines in question the decisive band is often 2 to 6 bits per weight.

The available quantizations in this band are primarily affine: uniform builds published by the mlx-community project, and hand-tuned mixed-bit-depth builds from the community at large (for the 397B model we compare against the most capable we could obtain, the "spicyneuron" 2.6-bit and 3.5-bit builds). The refined end of the affine family tunes its grids on data — GPTQ orders quantization by approximate second-order information from a calibration set [1], and AWQ scales channels by activation statistics [2].

Vector quantization of LLM weights is itself established in the CUDA ecosystem: GPTVQ interleaves Hessian-guided VQ with updates to the remaining weights [3], AQLM learns additive multi-codebook quantization on calibration data with end-to-end fine-tuning [4], and QuIP# combines incoherence processing with E8 lattice codebooks [5]. All three are calibration-dependent, and none publishes runnable artifacts for the Apple-Silicon MLX stack. The lineage of the method itself is older: codebooks fit by k-means [6] over subvectors is product quantization [7], here applied to weights with no data in the loop. To our knowledge, no vector-quantized artifacts have been published for this software stack at all — this paper contributes a ladder of them for each of three models, measured against the affine incumbents at matched bytes, under a fit that is entirely data-free where the methods above calibrate.

Notation. A VQ geometry is written dN/KM, where N is the subvector dimension and M the codebook size; builds in this paper range from d2/K16 to d8/K16384. For example, d4/K2048 groups weights into subvectors of 4 consecutive values and replaces each with an index into a 2048-entry codebook, storing $\log_2(2048)/4 = 2.75$ bits per weight. Every size in this paper is the measured size of the packed artifact on disk; every quality number is measured on the assembled model.

The method is deliberately minimal: per-tensor k-means over the weight subvectors, one flat codebook width across the whole surface, no data anywhere in the loop. Data-free matters for more than elegance. A calibration-fitted quantizer's quality is partly a property of its calibration set, so any evaluation resembling that set flatters it and any deployment unlike it is off-distribution. A weight-space fit has nothing to inherit: it cannot be tuned toward an evaluation even by accident, so its scores mean the same thing on any text. It also forces an honest measurement posture: when the quantizer never sees data, a quality number can only come from scoring the assembled artifact, and §4 shows that is the only trustworthy score anyway.

Claim 1 (method). At matched-or-smaller packed bytes, the data-free VQ builds beat the affine builds — hand-tuned mixed and uniform — on all three models (§3). The claim is fenced on both ends: the wins are measured from 1.75 to 5 bits per weight, and the crossover where the affine frontier passes above ours is bracketed at 4.5–6.0

bits on the dense model and 5.0–6.0 on the 35B MoE, measured from both sides in each case. At 8 bits affine is essentially lossless and there is nothing to beat. The quality advantage also carries a cost: VQ prefill throughput is roughly half that of the affine builds at 35B scale (§3.5).

Claim 2 (size targeting). A quantization can be tuned to a byte budget. Codebook widths land where log2(K)/d puts them — on the 397B the gap between adjacent widths is 31 GiB — and we make the axis continuous: a two-coefficient size model prices any target before the fit runs, and harvesting bits from the shallow layers, which tolerate them, sheds size at measured exchange rates (§3.4).

Claim 3 (measurement). Weight-space reconstruction error does not rank output quality here, and cannot steer design. We show this by construction: a pre-registered intervention improved precisely the weight-space statistic our mechanism analysis identified as the one that mattered, and the model got worse by 4.7x the effect it was built to fix (§4.3).

---

2. Method

The recipe has one moving part. In the mixture-of-experts models the expert tensors hold approximately 90% of the parameters; in the dense model the same role is played by the MLP trio — the three feed-forward projection matrices in each transformer layer (gate, up, and down), which together dominate a dense model's parameter count. These byte-dominant surfaces are the quantization target. A fixed non-expert skeleton is quantized affinely once and never varied, and the target tensors are replaced by a vector quantization whose dial is the geometry (d, K), held flat across every such tensor. A build is named by its geometry.

2.1 The skeleton

All 397B builds share one base: 6-bit structure, 4-bit attention projections, routers kept at bf16 (cheap structure is nearly free — demoting it 8→6 bits costs +0.0066

perplexity — while cheap routers are catastrophic at +11). The dense 27B builds splice VQ MLPs into a 4-bit affine conversion, carrying every other tensor through unchanged, which makes each build a controlled ablation of the MLP treatment against its base. The 397B's vision tower is kept at bf16 and grafted on last — exactly 912,020,960 bytes of tensor data, byte-identical across builds (the graft file itself is 912,057,227 B; the difference is the safetensors header) — and every size is stamped as measured before or after that graft.

2.2 The fit

For each target tensor independently: reshape the weights into d-dimensional subvectors, fit a K-entry codebook by k-means (k-means++ initialization, Lloyd iterations, per-group max-abs scales), store codes and codebook. Two properties matter downstream. Healthy reconstruction error scales with K — a fit at K=128 sits near 0.46 relative error and a healthy K=2048 fit near 0.19 — so acceptance thresholds are set per geometry. And the initialization subsamples the weights stochastically, so two fits of the same tensor differ; §2.6 measures the consequences and every comparison in this paper is read against them.

2.3 Packing

Codes are packed to their true bit-width after fitting; packing is bit-exact, verified at the logit level. Byte-aligned code widths are stored directly (packing them saves nothing and costs decode speed). All sizes are packed sizes measured on disk, and a row's size and its quality always come from the same artifact.

2.4 Size targeting

Flat geometries leave gaps between rungs. To reach a size inside a gap we harvest: hold the body geometry fixed and reduce K in the shallow layers (the first ten, at 397B scale), which tolerate cheap bits. The resulting size is predicted before the fit by a two-coefficient model — at 397B, `new = base − 1.87 GiB × shallow_bits`; on the dense 27B and the 35B, `total = code_bytes + scales + carry` with the carry measured once per model. Out-of-sample records for both forms are in §3.4.

2.5 The pipeline

Every artifact passed, in order: fit → reconstruction-error gate, run on a machine other than the one that fit it → pack → graft → structural verification → a smoke generation through the exact runtime the artifact ships with → scoring, both metrics, one instrument per model. The smoke generation is load-bearing: scoring exercises a prefill-shaped code path, serving exercises the fused decode kernels, and an artifact can score normally while being unable to serve — so nothing is fully validated until it has generated a token through the code path it ships with. Predictions are registered before numbers exist, with reading grids fixed in advance, so a wash cannot be reread afterwards as a win.

2.6 Instruments and noise floors

Four quantities appear throughout. **Perplexity (ppl)** is the exponentiated average negative log-likelihood of a fixed evaluation text under the model — lower is better, and a quantization's quality is read as its perplexity relative to other builds on the same text, never across texts. **KL divergence**, reported in millinats (mnats) — a *nat* is the unit of information measured in the natural logarithm, as a bit is in base 2, and a millinat is a thousandth of one — measures how far the quantized model's next-token probability distribution drifts from the full-precision model's, averaged over a fixed token stream; zero means the quantized model behaves identically. **Top-1 agreement** is the fraction of positions at which the quantized model's most probable token matches the full-precision model's. **Relative reconstruction error (relerr)** is a weight-space quantity — the norm of the difference between a tensor and its quantized reconstruction, relative to the tensor's norm — used only as a corruption gate, because §4.3 shows it does not rank output quality.

397B: streaming referee perplexity on frozen prose and code corpora, first 8192 tokens. Deterministic — an artifact reproduces its total negative log-likelihood to all printed decimals across launches and machines. **35B and 27B:** KL divergence to the bf16 model's cached logits, in millinats, with top-1 agreement, plus referee perplexity on the 27B — the same wikitext-style prose corpus at its first 2048 tokens; a single corpus, where two were used for the 397B. Comparator rows were re-verified on a

second machine and agree to every reported digit. The gemma-4 family, where we observed similar size-quality behavior, is excluded throughout: raw likelihood is invalid on those instruction-tuned models as a property of the model itself, scoring is therefore not deterministic, and no claim here rests on an instrument that cannot reproduce its own numbers.

Noise floors. Because unseeded fits are stochastic, two builds of identical geometry differ. We measured that spread where our comparisons live: dense 27B d2/K256, three draws — KL range 2.085 mnats, perplexity range 0.0447; 35B d2/K1024, two draws — 0.214 mnats; 397B d4/K256, two same-stack draws — 0.0256 prose perplexity; 397B d4/K2048, two draws — 0.0056 prose, 0.0104 code (the floor narrows as the codebook grows). The source of the width is the stochastic initialization: across draws, mean reconstruction error moves by 0.0001 while perplexity moves by 0.026 — the tail of the reconstruction moves, and the tail is what output quality responds to most strongly (§4.3). Every margin in §3 is stated as a multiple of the floor for its geometry, a margin inside its floor is reported as noise, and a floor is never borrowed across geometries silently — where a neighbouring geometry's floor stands in, the text says so and the multiple is read as a lower bound on confidence, not a measurement. The dense-family fitter gained a seed on 2026-08-22; the MoE fitters draw their initialization subsample unseeded by design. Every artifact in this paper is therefore a single unseeded draw with one exception: the 27B d2/K512 row is E142-27B arm 2, fit under a fixed seed after that date and reproducible bit-for-bit. Its margins are still read against a floor measured from unseeded draws, which makes them conservative — the floor contains draw variance the seeded arm does not have. That is why the floors above exist and why no margin is read without one.

---

3. Results

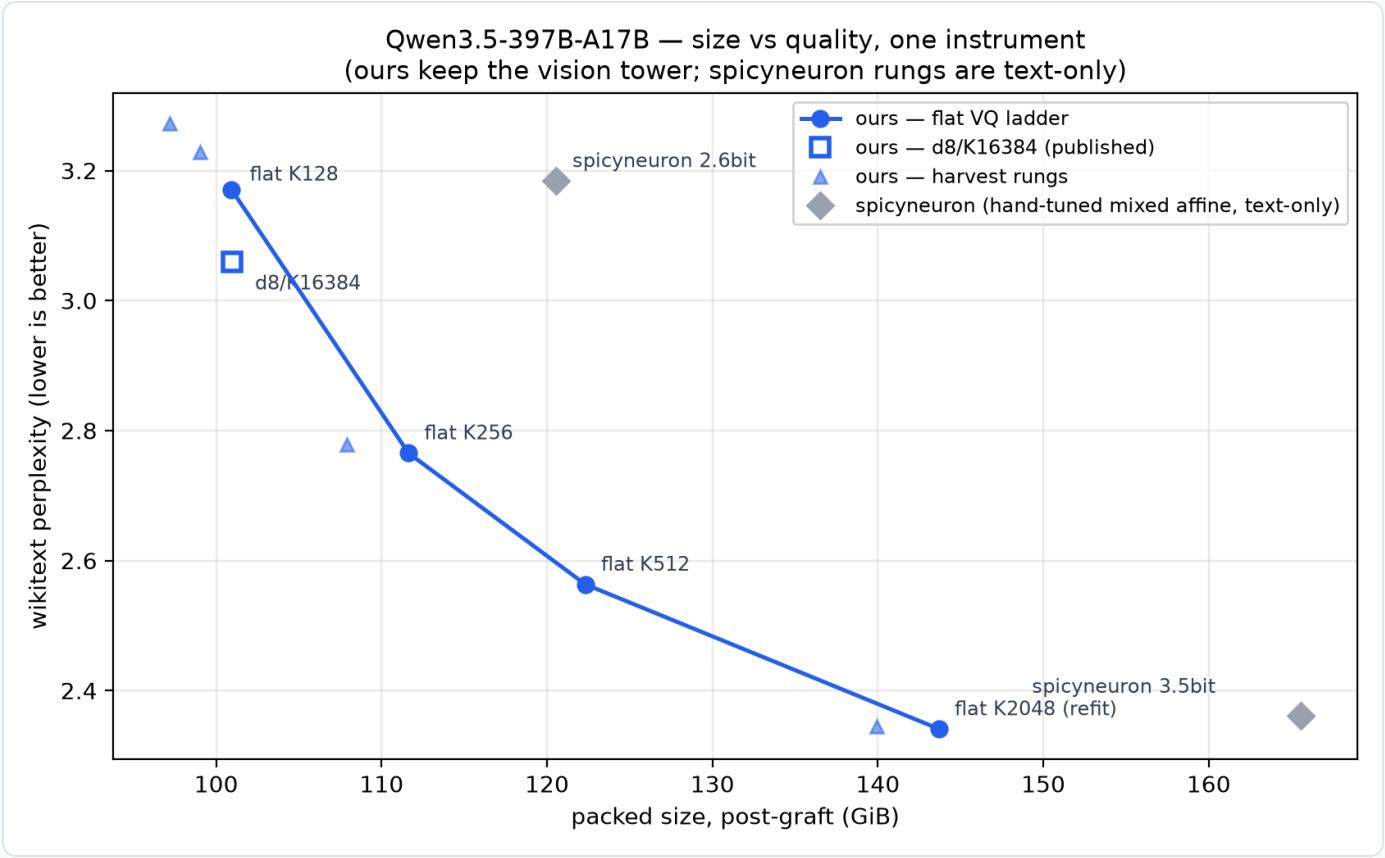
3.1 Geometry: what d and K buy

Rate is log2(K)/d, so the same bit rate is reachable with small vectors and small codebooks or large vectors and large codebooks. Measured at matched rate, exact twins within megabytes of each other:

RATE	PAIR	RESULT
2.00 bpw (35B)	d4/K256 vs d2/K16	d4 wins by 12.2% KL
3.00 bpw (27B)	d4/K4096 vs d2/K64	d4 wins by 8.6% KL
1.75 bpw (397B)	d8/K16384 vs d4/K128	d8 wins prose, 4.4x floor; code +0.0260 is 1.5x and does not clear the bar

Dimension pays at matched rate — consistently, and modestly, with the margin shrinking as the rate rises. It also has costs. d4 has a hard rate ceiling of 4.0 bpw (16-bit indices over 4 weights, even at a 65,536-entry codebook), so the high bands belong to d2. And large codebooks outgrow the GPU's fast on-chip memory: Apple's threadgroup limit is 32 KB, a d8/K16384 codebook is 256 KB, and serving it from device memory costs ~19% decode throughput (§3.5). The operational sweet spots this induces: d4 with the largest codebook that fits the band, d2 above 4 bpw, d8 where quality-per-byte justifies the decode tax.

3.2 The 397B ladder



397B ladder

Sizes below are whole-artifact post-graft bytes: what a user downloads. That convention is not symmetric here, and the asymmetry runs against us. Our artifacts carry the bf16 vision tower; the community comparators are text-only (2212 tensors, no tower), so each of our rows is 0.849 GiB heavier than a like-for-like comparison would make it. Every size margin we report against them is therefore understated by that amount — the d8 build's lead is 20.4 GiB rather than 19.6, and the flagship's 22.7 rather than 21.9. We keep the download-size convention and state the offset rather than restate the sizes, because a convention that gets adjusted in the reporter's favor is worth less than a conservative one.

Ours (VQ; prose / code perplexity, packed post-graft GiB):

BUILD	RELEASE	GIB	PROSE	CODE
flat d4/K128	—	100.93	3.1706	2.6988
flat d8/K16384	**VQ-2.2bpw**	100.97	3.0591	2.6728
flat d4/K256	**VQ-2.4bpw**	111.62	2.7655	2.6383

BUILD	RELEASE	GIB	PROSE	CODE
flat d4/K512	**VQ-2.6bpw**	122.31	2.5634	2.6123
flat d4/K2048	**VQ-3.1bpw**	143.68	2.3410	2.5963

Bold rows are published artifacts, downloadable at the sizes shown, under `TheDrainFlorist/Qwen3.5-397B-A17B-<release>`; the unbolded rung is a ladder point only. Release names are whole-artifact bits per weight — total bytes over parameter count, to one decimal — and so exceed the codebook rate, which covers only the quantized region: d4/K2048 stores 11-bit indices, 2.75 bpw of codes plus an fp16 scale per (row, 64). The measured column, not the name, is the number (§5).

Community affine (hand-tuned mixed allocation, text-only):

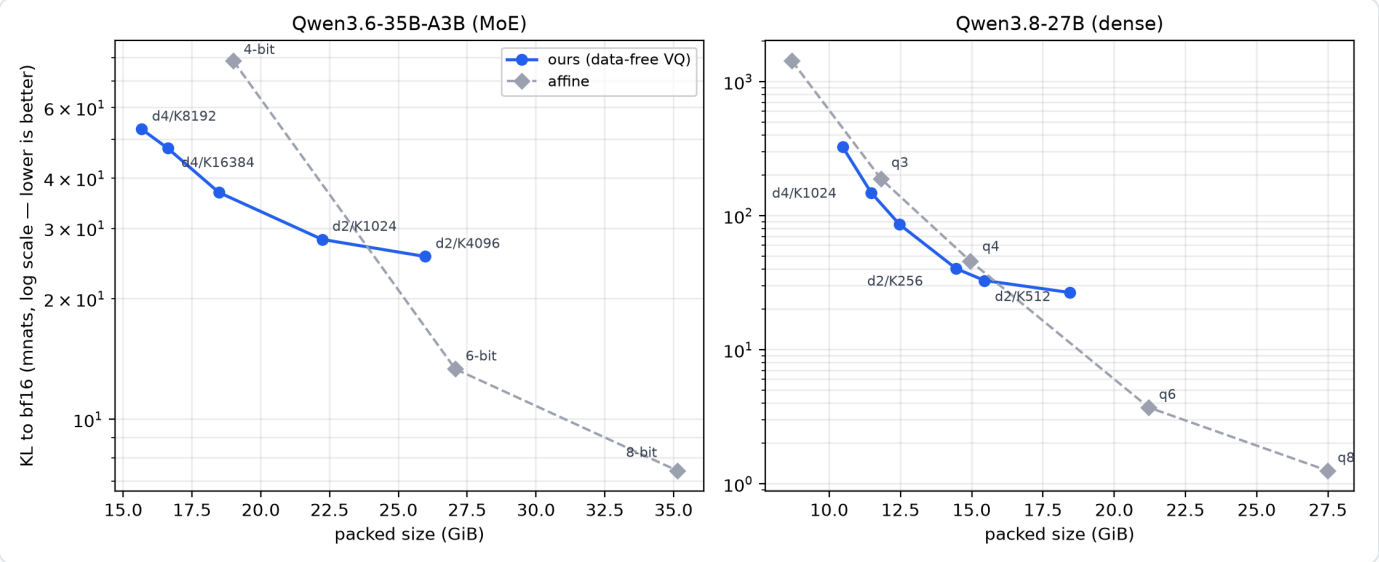
BUILD	GIB	PROSE	CODE
spicyneuron 2.6bit	120.57	3.1843	2.6667
spicyneuron 3.5bit	165.57	2.3614	2.6005

Three comparisons carry claim 1 here. **d4/K512 against the 2.6-bit community build:** at 1.7 GiB larger (0.9 GiB without the vision tower), prose perplexity is better by 0.6209 — 24 times the fit-to-fit floor — and code by 0.0544. No floor was measured at d4/K512 or d8; both here and in the d8 comparison below, the d4/K256 floor is borrowed, so these multiples are lower bounds on the margin rather than measurements of it. This is the closest size-matched pair on the ladder and the least ambiguous result in the paper. **d8/K16384 vs the same build:** at 19.6 GiB smaller, prose is better by 0.1252 (4.9x floor). **d4/K2048 vs the 3.5-bit community build:** 21.9 GiB smaller, with better prose perplexity by 0.0204 — 3.6 times this geometry's measured fit-to-fit floor of 0.0056 — and a code margin of 0.0042 that sits inside the 0.0104 code floor and is reported as a tie. The claim is therefore: smaller by 21.9 GiB, better on prose, tied on code — and 22.7 GiB smaller on the like-for-like basis described above.

Our ladder is monotone — no mixed-allocation build beats the flat rung at or above its own size, and a matched-byte sweep of allocation shapes at identical 141.42 GiB

spanned 0.32 perplexity with flat winning — which is why flat rungs are the reference points and mixed allocation is a size-targeting tool (§3.4), not a quality one.

3.3 The 35B MoE and the dense 27B



35B and 27B ladders

35B — ours (VQ):

BUILD	RELEASE	GIB	KL MNATS	TOP-1
d4/K8192	**VQ-3.8bpw**	15.67	53.02	89.55%
d4/K16384	—	16.61	47.54	89.81%
d2/K256	—	18.48	36.86	90.92%
d2/K1024	**VQ-5.4bpw**	22.23	28.14	92.22%
d2/K4096	—	25.98	25.50	92.52%

Bold rows are published artifacts, under `TheDrainFlorist/Qwen3.6-35B-A3B-<release>`. Two further 35B builds are published but do not appear on this flat ladder — a 13.79 GiB d4/K2048 rung below it, and an 18.71 GiB tail-weighted build that is not a flat geometry and so is not a point on this curve.

35B — affine:

BUILD	GIB	KL MNATS	TOP-1
4-bit (community)	19.00	78.56	85.61%
6-bit (ours)	27.07	13.36	94.65%
8-bit (community)	35.13	7.45	96.18%

One asymmetry in these 35B comparisons runs in our favor, and it is not visible in the sizes. Every affine comparator here quantizes the MoE router — the community 4-bit and 8-bit and our own 6-bit all carry quantized gate projections — while our VQ builds leave the routers at bf16. The bytes are trivial, 20 MiB or 0.14% of the artifact. But a router's output feeds an argmax over experts, so quantizing it can change *which* expert runs rather than perturbing an output proportionally, and the effect is therefore not bounded by the byte share the way an ordinary precision difference would be. We have not measured it: that would take a VQ build with routers forced to 8 bits, re-scored, and we did not run one. The 397B comparisons in §3.2 are unaffected — there our builds and both comparators keep routers at bf16.

Every 35B size above includes the 333-tensor bf16 vision tower, which the community comparators ship and our builds now carry: 0.832 GiB, byte-identical across builds, unquantized in all of them. Every row is a measured artifact; comparisons here are like-for-like at face value. (The 397B in §3.2 sits the other way round: our builds carry a tower its comparators lack, which is why that section states an offset rather than applying one.)

At the small end VQ dominates: 47.5 mnats at 16.6 GiB against affine's 78.6 at 19.0 — 39% less divergence in 2.4 GiB fewer bytes. At 5 bits per weight the comparison becomes a placement rather than a dominance: d2/K1024 lands between two affine rungs, and two independent fits of it score 28.14 and 27.93 against 38.7 for the affine frontier log-interpolated to the same size — both draws a factor of ~1.4 below the line, roughly 50x the draw floor. One rung higher the sign flips: at 6 bpw, d2/K4096 is 1.1 GiB smaller than the 6-bit affine build and scores 1.91x worse — 57x the floor (the d2/K1024 floor, borrowed: no d2/K4096 floor was measured; in the one case where we measured floors at two K on one family, 397B d4, the larger K's floor was 4.6x narrower, so the borrow likely understates the multiple), conclusive. (That two

"6-bit" artifacts differ by 1.1 GiB is expected: a nominal rate names the code width on the quantized surface, while total bytes include each method's scale overhead and its treatment of the non-expert remainder — which is why every comparison in this paper is by measured size, never by nominal rate.) **The crossover on this family sits between 5.0 and 6.0 bits per weight.**

27B — ours (VQ):

BUILD	RELEASE	GIB	KL MNATS	TOP-1	PPL
d4/K256	–	10.47	325.6	76.5%	6.403
d4/K1024	–	11.47	148.5	82.5%	5.525
d4/K4096	**VQ-3.9bpw**	12.47	85.8	86.1%	5.229
d2/K256	**VQ-4.5bpw**	14.45	40.3	90.1%	5.233
d2/K512	**VQ-4.8bpw**	15.45	32.8	90.8%	5.162
d2/K4096	–	18.44	26.7	91.7%	5.242

Bold rows are published artifacts, under `TheDrainFlorist/Qwen3.8-27B-<release>`.

27B — affine. Unlike the 397B and 35B comparators, these rungs are our own conversions.

BUILD	GIB	KL MNATS	TOP-1	PPL
q2	8.69	1426.9	46.1%	16.435
q3	11.82	187.8	79.5%	5.832
q4	14.95	45.8	89.8%	5.206
q6	21.21	3.71	96.8%	5.260
q8	27.48	1.25	98.5%	5.241

Every 27B size above includes the 333-tensor bf16 vision tower (0.858 GiB), grafted onto rungs and comparators alike. The offset is uniform, so differences carry over unchanged; ratios do not.

The recipe is not an MoE phenomenon. Below 4.5 bpw every VQ point sits above the affine line at its size: d4/K1024 beats q3 on both metrics at 0.35 GiB less, and d2/K512 beats q4 by 28.4% KL (6.2x floor) and +1.02 points top-1 at q4-class size. Above, the picture inverts: q6 beats our best 6-bpw rung by 7.2x KL at 2.8 GiB more. **The dense crossover is bracketed at 4.5–6.0 bpw — nearly the same band as the MoE.** One instrument note: the perplexity column barely moves from q3 upward (5.21–5.26 across the affine rungs above q3, a span of 0.054 against a 0.0447 floor) while KL moves 37x. On this instruction-tuned family, perplexity cannot rank quantizations; divergence and agreement can.

3.4 Size targeting

The size models' out-of-sample record: at 397B, six hits and one in-band across seven predictions (worst miss 0.4 GiB, best +0.02); on the 35B, three consecutive geometry predictions at −0.03%, −0.30% and −0.37%; on the dense 27B, three builds across two geometries within 0.003 GiB. Pricing a build before fitting it works on every model we tried it on.

Harvest exchange rates, measured at three base richnesses on the 397B (prose perplexity per GiB shed): 0.0315 from a K128 base, 0.0033 from K256, 0.0011 from K2048 — the cost falls ~30x as the base gets richer, and is roughly half the cost of stepping down the flat ladder. The fence: harvest cost is monotone at every base measured, and no harvest build beats the flat rung at the flat rung's own size. What harvest buys is the sizes in between. Together with the size model, the capability is: name a byte budget, price the build, fit it once.

3.5 Runtime performance and kernel support

None of this serves without custom Metal kernels: a fused decode-and-matmul path that reads codes and codebook directly (per-K bit-width extraction in-kernel), a device-memory codebook variant for the codebooks that exceed Apple's 32 KB threadgroup memory, and a zero-copy view that dispatches byte-aligned unpacked codes through the packed kernel (+25–33% prefill, bit-exact). All kernel variants are

accepted only on bit-identity with a reference path where both load, and on relative error against a float32 reference where only one does.

Decode throughput is equivalent across the d4 geometries, whose codebooks fit in threadgroup memory. It is not equivalent where they do not: d8/K16384's 256 KB codebook streams from device memory and costs approximately 19% of decode throughput against its same-size d4 sibling — the measured price of the quality its geometry buys, and one that may differ on hardware with a different memory hierarchy. Against affine, VQ prefill remains ~0.5x at 35B scale even after the zero-copy dispatch described above (the +25–33% prefill recovery); decode is within 10–20%. The asymmetry is the signature of where each phase's time goes. Decode generates one token at a time and is bound by memory bandwidth — the cost of reading the weights — and a VQ artifact has fewer bytes to read, so the extra arithmetic of in-kernel codebook lookups hides behind the memory traffic and decode stays near parity on this hardware — the balance is set by the machine's bandwidth-to-compute ratio, and a machine with less memory bandwidth will sit elsewhere on it. Prefill processes the whole prompt as large matrix multiplies and is bound by arithmetic throughput; there the same per-weight decode work is added to a compute-saturated path with no bandwidth saving to pay for it, and it surfaces as the 2x gap. This is an interpretation consistent with the measured split rather than a profiled attribution; what is measured is the pair of ratios. Speed numbers here are same-session ratios between arms: we found decode throughput at ~100 GiB residency to be bimodal on our hardware (the same artifact varying 40% run to run, with swap, thermals and storage path each ruled out by measurement), we have not characterized other sizes, and we therefore publish no absolute throughput figures. Three further speed levers were tested and closed (fused row-gather, byte-aligned packing, native-bf16 kernels), each with a measured null or negative effect in the lab record; distillation-based refinement at 397B/2-bit was falsified outright.

---

4. Negative results

This section reports what did not work, what cannot be reached, and what those boundaries imply. They are results, not caveats: each was measured, and several bound the claims of §3.

4.1 The 8-bit ceiling

At 8 bits affine is essentially lossless: 7.4 mnats on the 35B, and 1.25 on the 27B — measured on a uniform 8-bit conversion with no per-module overrides (27.48 GiB), built the same way as every other rung on its ladder. Extrapolating the 27B's measured slope to that target puts 8-bit-class quality at roughly 27 bits per weight.

Nothing we measured approaches that ceiling under the byte budgets where VQ wins. On the 27B, the ladder's own slope says why: divergence falls by x0.673 per added bit near 4.5 bpw but only x0.868 per bit by 6.0, so the measured slope has to be carried a very long way to reach a near-lossless target. The 35B agrees from the other side: a 6-bit affine build inside a 28 GiB budget misses 8-bit quality by 1.8x, and our 5-bit build misses it by 3.8x. On both models, 8-bit quality costs 8-bit bytes, for affine and for VQ alike.

4.2 Where the geometry axes stop paying

Dimension pays at matched rate (§3.1) but the margin shrinks as rate rises — 12.2% at 2.0 bpw, 8.6% at 3.0 — and at 4.0 bpw it stops paying cleanly. A d4/K65536 rung matched against a d2/K256 rung 0.1 GiB smaller (14.55 vs 14.45) splits: divergence favors d4 by 2.2 mnats, which is 1.06 times a floor measured at a different geometry and so not a margin we read, while perplexity favors d2 by 0.078, or 1.75 times that same borrowed floor. Neither pre-registered branch fired; the honest reading is a wash leaning d2, and the dimension advantage is not established above 3 bpw. Codebook size pays with the expected diminishing returns: on the 35B d4 line, each doubling of K buys less (17.0, then 15.5, then 5.5 mnats). Harvest never beats the flat rung at its own size, at any base richness we measured; its value is reachability, not quality. And calibration lost on its home turf where we tested it: an activation-calibrated method (a DWQ-style distilled scale fit in the MLX toolchain [8]) fell to

uniform quantization on the dense 27B, and per-layer sensitivity probes rank layers in ways that do not survive contact with assembled-model scores on MoE.

4.3 Reconstruction error cannot steer design

The central negative, shown by construction. A refit of one published geometry scored worse than the original at byte-identical size while having *lower* reconstruction error on every projection. Percentile analysis located the trade: the refit was better where most weights live and worse precisely in the top 0.1% by magnitude — and mean reconstruction error, a bulk statistic, reported the trade as an improvement. The mechanism replicated across 36 tensors and has a clean cause: body-layer weights are sub-Gaussian, so a better-average-distortion codebook is bought from the tail, and the tail is what output quality responds to.

So we engineered the converse as a designed test, pre-registering the reading before any number existed: reweight the k-means objective to recover exactly that tail band. It worked in weight space — the tail error bands improved as designed — **and the resulting model was 4.7 times worse than the regression the change was designed to repair.** At fine-grained fits the two metrics track (improving the objective at a 0.08-relative-error geometry improved the model, 2.8x the floor); where centroids are scarce they invert; and no weight-space statistic we measured predicts which side of that line a fit lands on. Only the assembled model knows.

Two scope notes. These comparisons are between arms sharing the same base weights and differing only in the fitter, so they are unaffected by any difference in when, or on what software stack, a build was fit. And the same phenomenon sets the fit-to-fit noise floors of §2.6: across stochastic draws, mean reconstruction error is essentially constant while output quality moves by more than several margins we had been prepared to report. Any comparison at that scale is a comparison of draws.

---

5. Measurement discipline

The results above were produced under recording rules adopted early in the project, after mechanical checks — not inspection — caught each of the first wrong numbers. They are stated because the tables cannot be interpreted without them, and because several candidate claims from earlier drafts did not survive them.

Predictions and their reading grids are written before fitting or scoring, and falsified predictions are recorded as falsified. A margin is quoted as a multiple of the measured fit-to-fit floor for its own geometry (§2.6); where a neighbouring geometry's floor stands in, that is disclosed and the multiple is read as a lower bound; a margin inside its floor is noise regardless of direction. Applying this rule retrospectively retired three of this paper's own candidate claims.

A comparison row names the artifact and instrument that produced it. A number older than the artifact it faces is re-measured rather than cited; a row's size and quality come from the same artifact, with size as packed bytes on disk. Artifacts are gated before scoring, on a machine other than the one that fit them, and a gate's verdict is trusted only after the gate has failed on a known-bad input and passed on a known-good one. No artifact is treated as releasable until it has generated tokens through the exact runtime it ships with.

Each artifact's shards are fingerprinted — byte count, modification time, and a hash of the shard's head, stored outside the artifact — enough to identify a shard and to catch a silent rewrite, though not to certify every byte. Stored metadata is treated as a record of intent rather than of content. A checkpoint's `model_type` field, for example, names the loader code path rather than the model: Qwen 3.6 and 3.8 share the 3.5 architecture, so their checkpoints all declare a `qwen3_5` variant, and every derived build inherits the stamp. A field like that cannot identify what a file contains, so any stored property that carries a claim is verified against the bytes themselves — a tensor compared with its claimed base, a hash recomputed, a flag traced to the code that reads it.

None of this is novel; it is ordinary verification discipline, applied in a setting where the wrong numbers are the plausible ones.

6. Limitations

Coverage. Three models from one vendor family, only one of them dense; a second dense model would test whether the 27B generalizes. (The gemma-4 family showed similar behavior but cannot be scored deterministically and is excluded.) Single software stack (MLX/Metal); the kernel conclusions — threadgroup capacity, the d8 decode tax — are specific to Apple Silicon.

Unmeasured regions. The VQ/affine crossover is bracketed on the 35B and the 27B, but not on the 397B: affine builds above 3.5 bits exist or could be produced for that model, but at ~225 GB for a 4-bit build and ~320 GB for 6-bit they exceed the memory of any machine available to this project, so whether the same crossover band holds at that scale is untested. Whether dimension still pays at d4's 4.0 bpw ceiling was tested and did not resolve: the d4/K65536 rate twin (§4.2) came out a wash leaning d2, so the dimension advantage is not established above 3 bpw. No dense harvest rung has been built: claim 2's exchange rates are measured on MoE only, and the mechanism we propose (shallow-layer redundancy) predicts they should weaken on dense — a prediction, not a result.

Instrument limits. The 397B noise floors rest on two draws per geometry (0.0256 prose and ~0.0178 code at d4/K256; 0.0056 prose and 0.0104 code at d4/K2048 — the floor narrows substantially as the codebook grows). The 35B floor bounds initialization variance only (same box, same geometry). Perplexity cannot rank quantizations on the instruction-tuned 27B (§3.3). Decode throughput was bimodal at ~100 GiB residency on our 128 GB machine and is uncharacterized at other sizes; we publish ratios within a session, never absolutes.

Costs we pay. Prefill remains ~0.5x affine at 35B scale even after the shipped lever. Codebooks beyond threadgroup capacity pay ~19% decode. One published artifact cannot be rebuilt at all: the 111.6 GiB d4/K256 build (`VQ-2.4bpw`) predates the manifest system, and the log recording its exact fit invocation was overwritten four days after the fit, so its remaining unrecoverable inputs are the fit flags themselves. It remains downloadable, its scores reproduce exactly on the artifact, and its quality sits

inside the measured draw distribution of its geometry — a favorable but unexceptional draw, not a mystery (§2.6's floors are how we know).

---

7. Reproducibility

All artifacts are published under `TheDrainFlorist` on Hugging Face with their VQ runtimes bundled in-checkpoint (stock mlx-lm, no patches). Where a repository's weights were upgraded in place, the previous build remains fetchable at its pinned revision and the card labels which weights produced which benchmark rows. Published artifacts carry external manifests. With the single exception noted in §2.6, the fits behind them are unseeded single draws, so a published build is reproducible in recipe and geometry but not bit-for-bit; that is precisely why every margin in this paper is quoted against a measured fit-to-fit floor rather than against a repeated build. The dense-family fitter has since gained a seed.

Which copy of a runtime executes is environment-dependent, so runtime-dependent claims name the resolved, executing copy rather than a file believed to be loaded. The bundled runtimes here are verified three ways: hash-compared against the runtime that produced the published scores, exercised as the executing copy in a stock environment by generating through the shipping kernel, and passed kernel acceptance as the unit under test lifted from the artifact itself. Fit, pack, verify, gate and scoring scripts are published in the project repository, **VQLab** (github.com/noahzelezny/VQLab, Apache-2.0). The prose referee corpus ships with it; the code corpus is drawn from a private codebase and does not ship — every code-perplexity number is a relative comparison between builds on that same fixed text, and its construction is described in the repository. Nothing was fit on data, so there is no train/eval overlap to disclose.

References

[1] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.* arXiv:2210.17323, 2022.

[2] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han. *AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration.* arXiv:2306.00978, 2023.

[3] M. van Baalen, A. Kuzmin, I. Koryakovskiy, M. Nagel, P. Couperus, C. Bastoul, E. Mahurin, T. Blankevoort, P. Whatmough. *GPTVQ: The Blessing of Dimensionality for LLM Quantization.* arXiv:2402.15319, 2024.

[4] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, D. Alistarh. *Extreme Compression of Large Language Models via Additive Quantization.* ICML 2024; arXiv:2401.06118.

[5] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, C. De Sa. *QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks.* arXiv:2402.04396, 2024.

[6] S. Lloyd. *Least Squares Quantization in PCM.* IEEE Transactions on Information Theory 28(2):129–137, 1982.

[7] H. Jégou, M. Douze, C. Schmid. *Product Quantization for Nearest Neighbor Search.* IEEE Transactions on Pattern Analysis and Machine Intelligence 33(1):117–128, 2011.

[8] A. Hannun, J. Digani, A. Katharopoulos, R. Collobert. *MLX: Efficient and Flexible Machine Learning on Apple Silicon.* github.com/ml-explore/mlx, 2023; DWQ as implemented in mlx-lm.

Acknowledgments

The spicyneuron and mlx-community builds served as comparators throughout; this work exists because those artifacts were public, pinned, and worth measuring against. We hope ours are the same.

AI disclosure. The experiments in this work were executed with substantial assistance from large language model agents (Anthropic Claude, Opus and Sonnet-class models), which operated the fitting, packing, verification, and scoring pipelines under the author's direction, and assisted in drafting this manuscript. All quantitative results were produced by the deterministic instruments described in §2.6, are traceable to a committed laboratory record, and were verified independently of any model-generated summary. The author directed all experiments, made all methodological decisions, and takes sole responsibility for the content.